

Secure Email Gateway Configuration

SpamAssassin, ClamAV, and DLP Integration Guide

Deliverable 4 of 6

Table of Contents

1. Gateway Overview
2. SpamAssassin Deployment
3. ClamAV Anti-Malware Integration
4. DLP Implementation
5. Gateway Architecture
6. Operational Recommendations
7. Conclusion

1. Gateway Overview

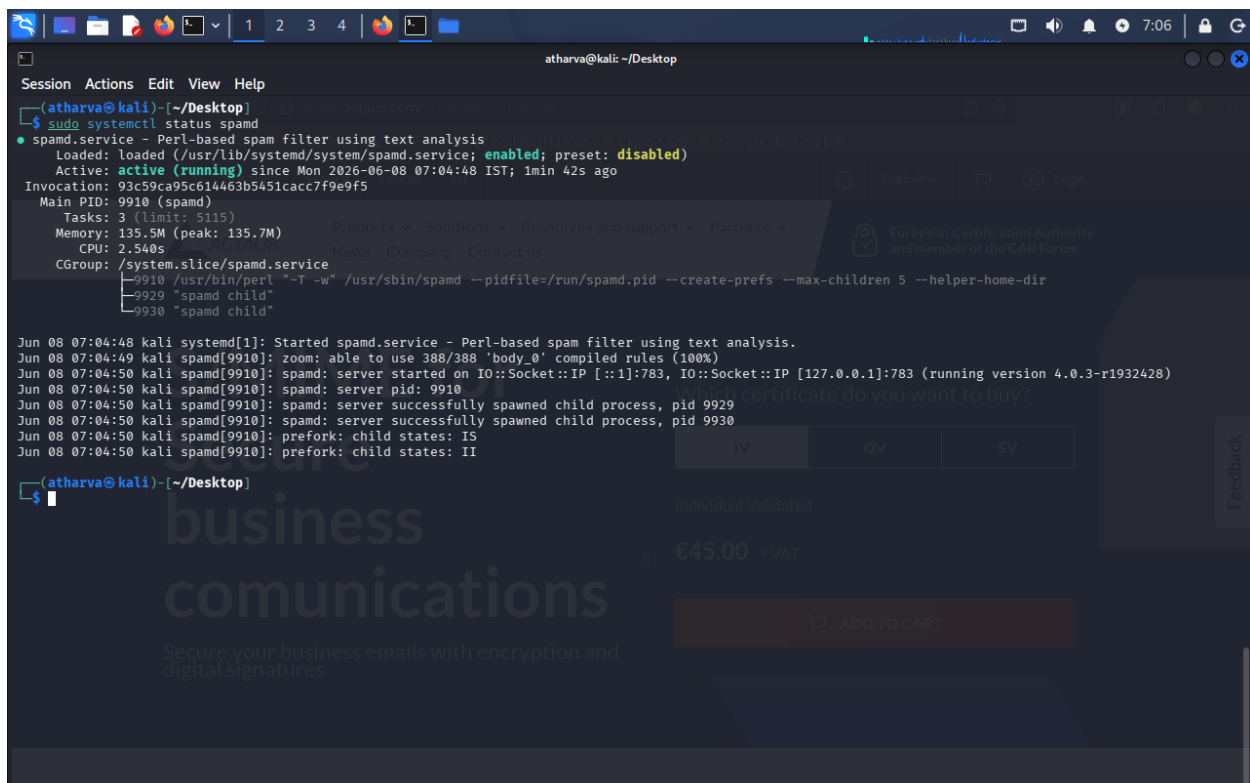
A secure email gateway sits between the public internet and the internal mail server. Every message - inbound or outbound - passes through it before reaching its final destination. This is the single best place to apply content filtering, malware scanning, and data loss prevention because all the email traffic is funnelled through one inspection point.

For this project, the gateway was built from three open-source components, each tackling a different category of threat: SpamAssassin for unwanted commercial mail and phishing, ClamAV for malware in attachments, and a custom Python module for data loss prevention.

2. SpamAssassin Deployment

2.1 Installation

SpamAssassin was installed from the Kali repositories and started as a systemd service. The service status output confirms the daemon is running and accepting connections.

A screenshot of a Kali Linux terminal window. The terminal shows the command `sudo systemctl status spamd` and its output. The output indicates that the `spamd.service` is loaded, active (running), and has been running since Mon 2026-06-08 07:04:48 IST. It shows the main PID as 9910 and lists tasks, memory usage, and CPU usage. The CGroup section shows the service's process tree. Below the service status, there are logs showing the service starting, spawning child processes (9929, 9930), and the prefork child states (IS, II). The terminal window has a dark theme and a window title bar at the top.

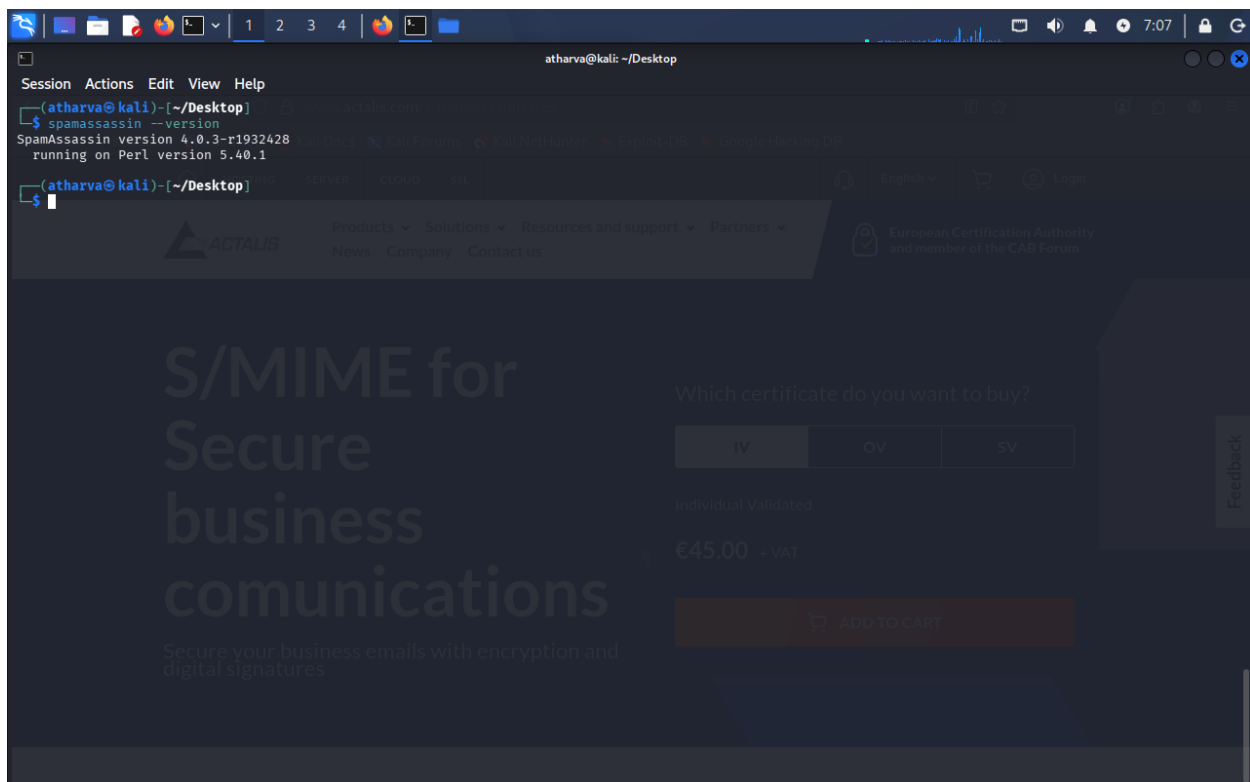
```
atharva@kali: ~/Desktop
Session Actions Edit View Help
(atharva@kali)~$ sudo systemctl status spamd
● spamd.service - Perl-based spam filter using text analysis
   Loaded: loaded (/usr/lib/systemd/system/spamd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-06-08 07:04:48 IST; 1min 42s ago
 Invocation: 93c59ca95c614463b5451cacc7f9e9f5
    Main PID: 9910 (spamd)
      Tasks: 3 (limit: 5115)
   Memory: 135.5M (peak: 135.7M)
      CPU: 2.540s
   CGroup: /system.slice/spamd.service
            └─9910 /usr/bin/perl "-T -w" /usr/sbin/spamd --pidfile=/run/spamd.pid --create-prefs --max-children 5 --helper-home-dir
              └─9929 "spamd child"
                └─9930 "spamd child"

Jun 08 07:04:48 kali systemd[1]: Started spamd.service - Perl-based spam filter using text analysis.
Jun 08 07:04:49 kali spamd[9910]: zoom: able to use 388/388 'body_0' compiled rules (100%)
Jun 08 07:04:50 kali spamd[9910]: spamd: server started on IO::Socket::IP [::1]:783, IO::Socket::IP [127.0.0.1]:783 (running version 4.0.3-r1932428)
Jun 08 07:04:50 kali spamd[9910]: spamd: server pid: 9910
Jun 08 07:04:50 kali spamd[9910]: spamd: server successfully spawned child process, pid 9929
Jun 08 07:04:50 kali spamd[9910]: spamd: server successfully spawned child process, pid 9930
Jun 08 07:04:50 kali spamd[9910]: prefork: child states: IS
Jun 08 07:04:50 kali spamd[9910]: prefork: child states: II

(atharva@kali)~$
```

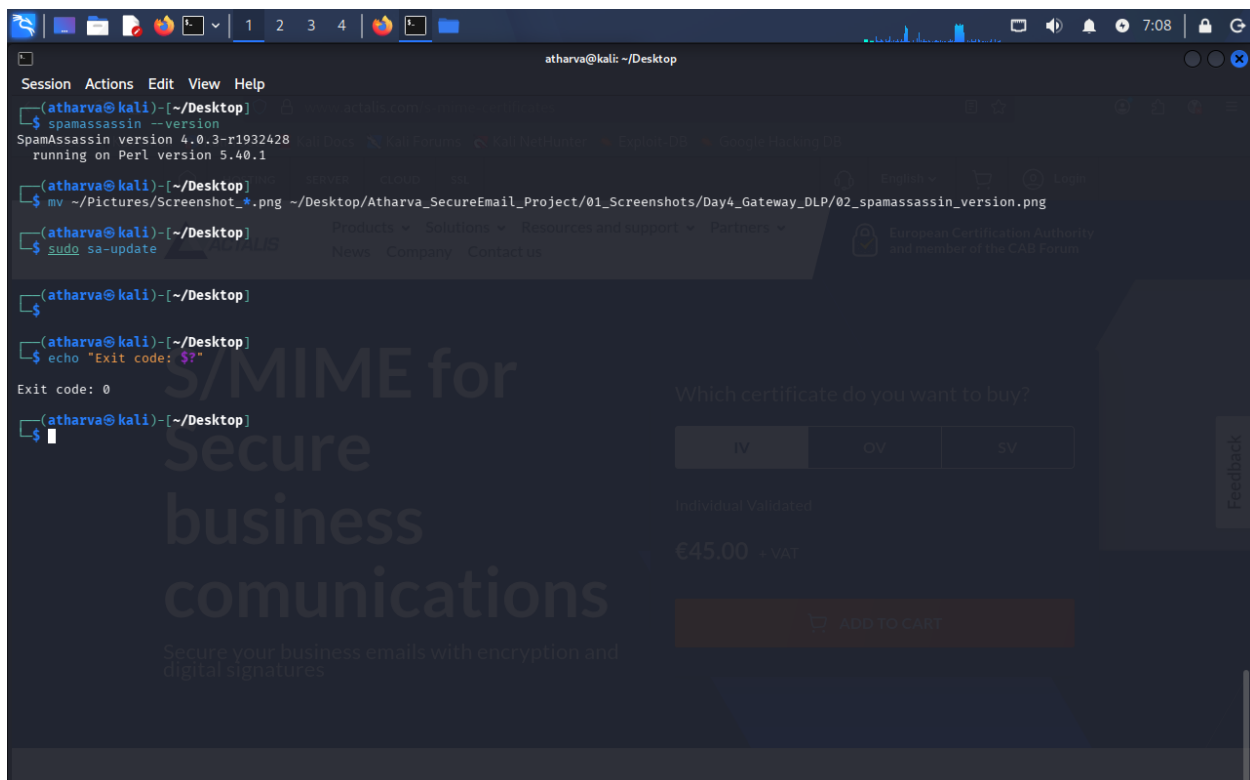
2.2 Rule Updates

The `sa-update` command fetches the latest community-maintained spam rules. These rules contain heuristics, regex patterns, and Bayesian filters tuned against current spam campaigns.



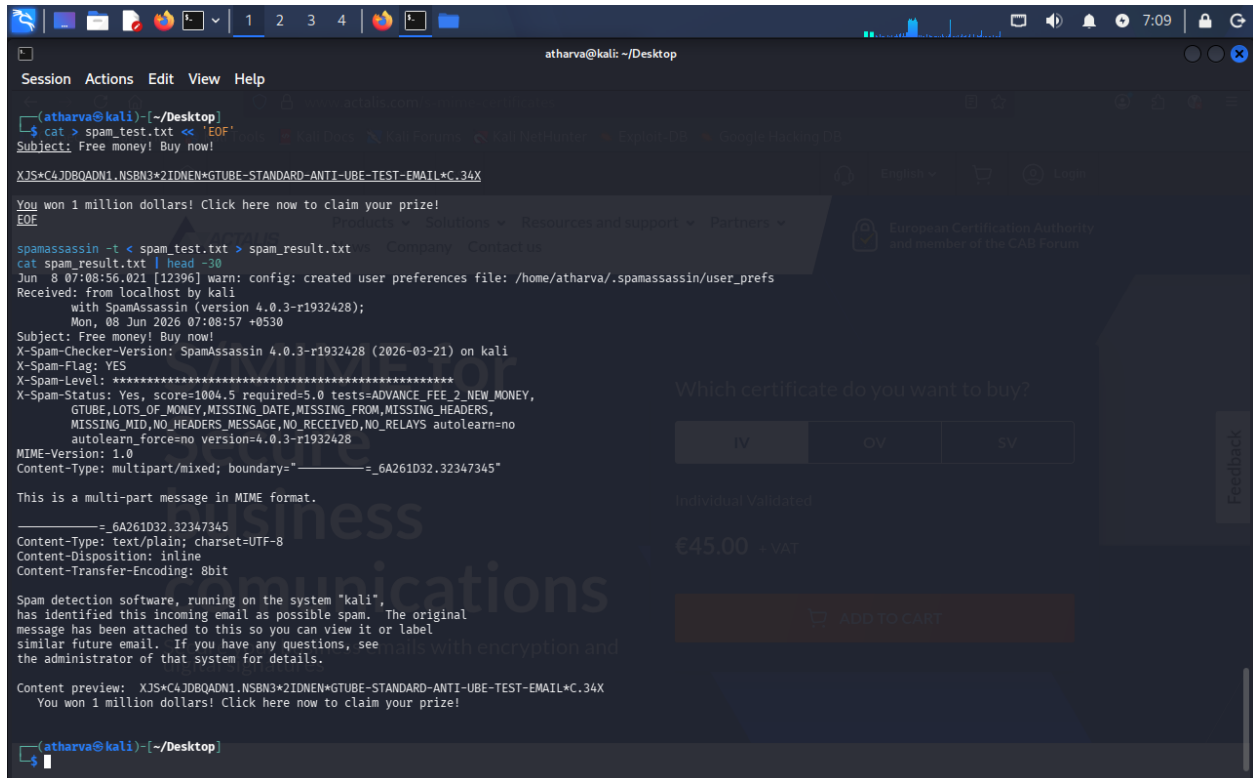
2.3 Spam Detection Test

The GTUBE string (Generic Test for Unsolicited Bulk Email) is a standard signature that SpamAssassin always classifies as spam. A test message containing GTUBE was created and scored. The result shows a very high spam score and the X-Spam-Flag header set to YES.



2.4 Normal Email Test

To prove the filter does not over-block legitimate mail, a normal business email was also scored. The score is well below the spam threshold (typically 5.0) and no spam flag is added.



```
(atharva@kali)~[/Desktop]
$ cat > spam_test.txt << 'EOF'
Subject: Free money! Buy now!

XJS+C4J0BQADN1.NSBN3+2IDNEN+GTUBE-STANDARD-ANTI-UBE-TEST-EMAIL+C.34X

You won 1 million dollars! Click here now to claim your prize!
EOF
$ spamassassin -t < spam_test.txt > spam_result.txt
$ cat spam_result.txt | head -30
Jun  8 07:08:56.021 [12396] warn: config: created user preferences file: /home/atharva/.spamassassin/user_prefs
Received: from localhost by kali
        with SpamAssassin (version 4.0.3-r1932428);
        Mon, 08 Jun 2026 07:08:57 +0530
Subject: Free money! Buy now!
X-Spam-Checker-Version: SpamAssassin 4.0.3-r1932428 (2026-03-21) on kali
X-Spam-Flag: YES
X-Spam-Level: *****
X-Spam-Status: Yes, score=1004.5 required=5.0 tests=ADVANCE_FEE_2_NEW_MONEY,
GTUBE,LOTS_OF_MONEY,MISSING_DATE,MISSING_FROM,MISSING_HEADERS,
MISSING_MID,NO_HEADERS_MESSAGE,NO_RECEIVED,NO_RELAYS autolearn=no
autolearn_force=no version=4.0.3-r1932428
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="=_6A261D32.32347345"

This is a multi-part message in MIME format.

-----=_6A261D32.32347345
Content-Type: text/plain; charset=UTF-8
Content-Disposition: inline
Content-Transfer-Encoding: 8bit

Spam detection software, running on the system "kali",
has identified this incoming email as possible spam.  The original
message has been attached to this so you can view it or label
similar future email.  If you have any questions, see
the administrator of that system for details.

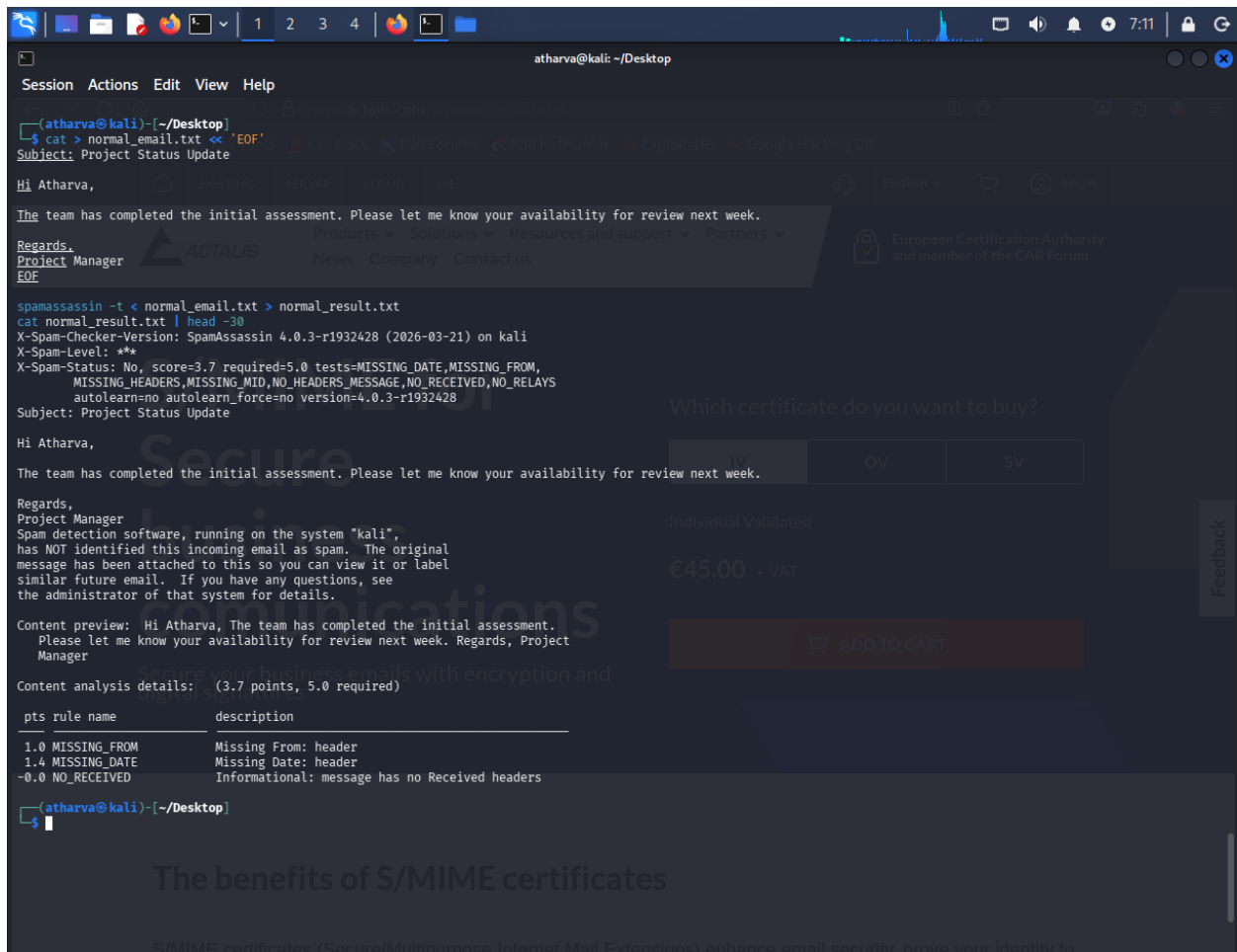
Content preview:  XJS+C4J0BQADN1.NSBN3+2IDNEN+GTUBE-STANDARD-ANTI-UBE-TEST-EMAIL+C.34X
You won 1 million dollars! Click here now to claim your prize!

(atharva@kali)~[/Desktop]
$
```

3. ClamAV Anti-Malware Integration

3.1 Installation and Updates

ClamAV was installed alongside the freshclam updater service. Freshclam pulls down the latest virus signature database, currently containing over 8 million known malware fingerprints.



3.2 EICAR Test Detection

EICAR is the industry-standard test signature - it is not actual malware but every antivirus product is required to detect it. A file containing the EICAR string was scanned and ClamAV correctly identified it as Win.Test.EICAR_HDB-1.

```
atharva@kali: ~/Desktop
Session Actions Edit View Help
(atharva@kali)~[~/Desktop]
$ echo 'X50!P%@AP[4\PZX54(P^)7CC)7]$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > test_virus.txt

(atharva@kali)~[~/Desktop]
$ clamscan test_virus.txt

Loading: 21s, ETA: 0s [=====] 3.63M/3.63M sigs
Compiling: 3s, ETA: 0s [=====] 41/41 tasks

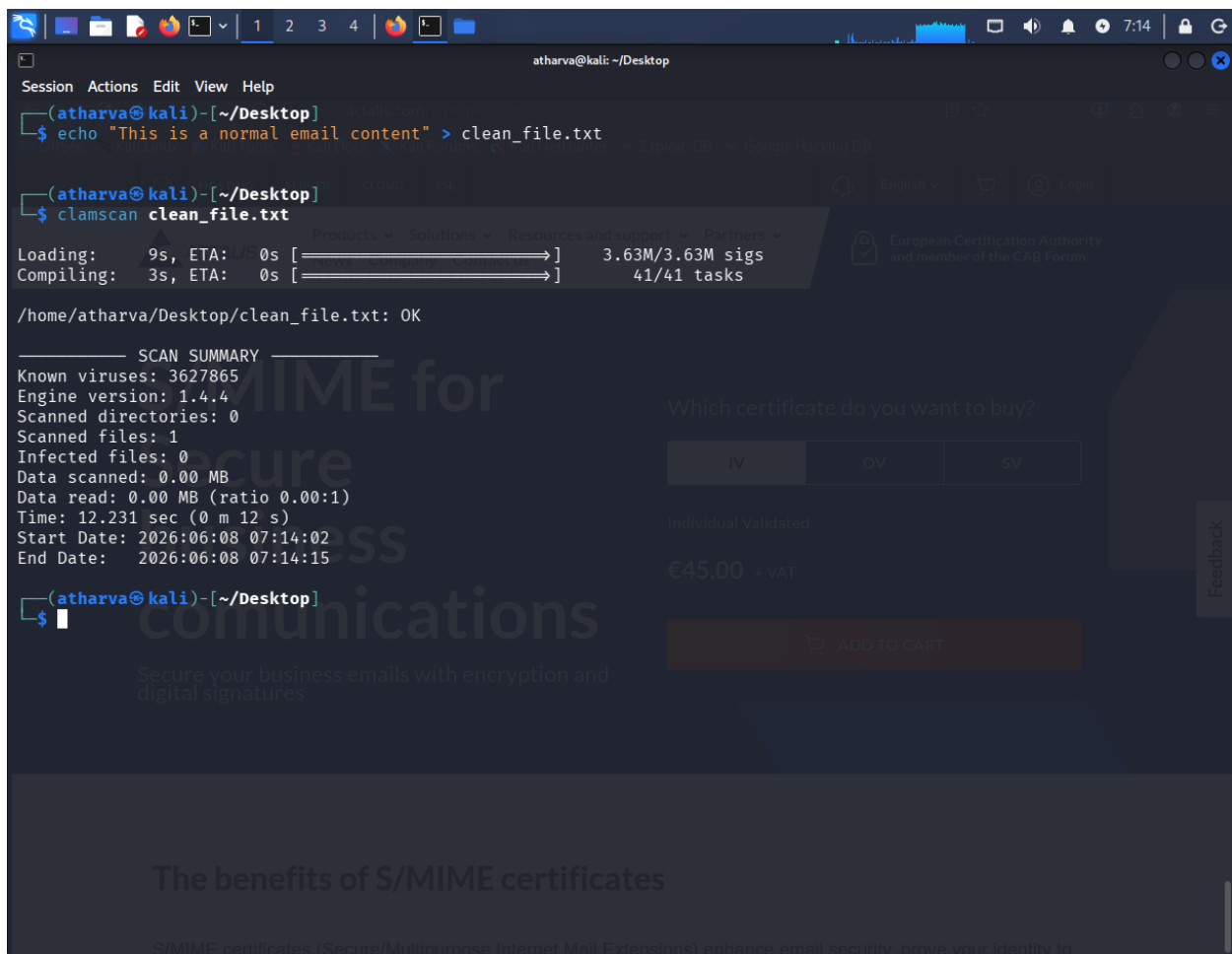
/home/atharva/Desktop/test_virus.txt: Eicar-Signature FOUND

----- SCAN SUMMARY -----
Known viruses: 3627865
Engine version: 1.4.4
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 24.161 sec (0 m 24 s)
Start Date: 2026:06:08 07:13:04
End Date: 2026:06:08 07:13:28

(atharva@kali)~[~/Desktop]
$
```

3.3 Clean File Test

As with SpamAssassin, the filter was also verified against a clean file to confirm it does not produce false positives.



4. DLP Implementation

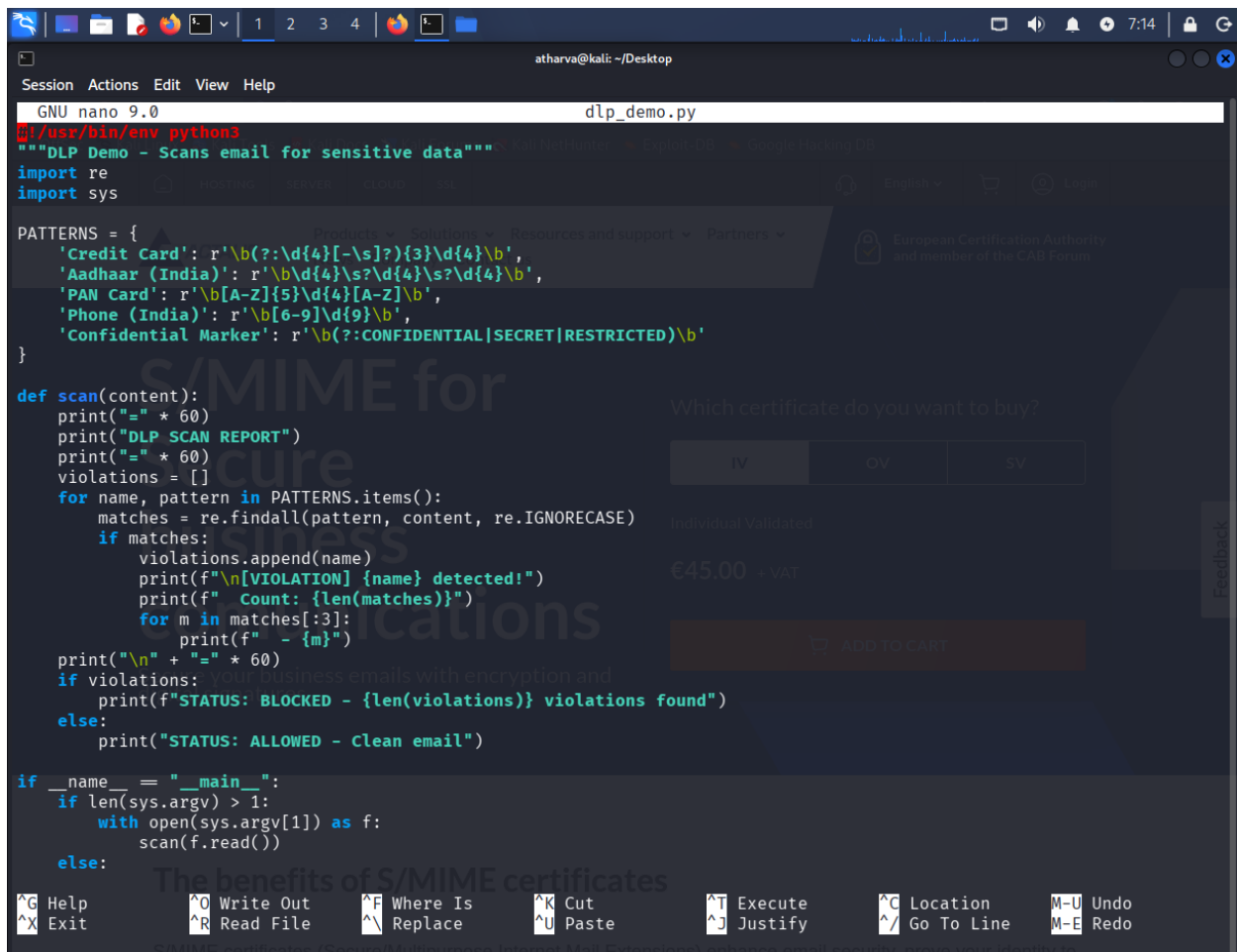
Data Loss Prevention prevents sensitive information from leaving the organisation through outbound email. A custom Python script was written to demonstrate the concept, covering patterns commonly seen in Indian corporate environments.

4.1 Patterns Covered

- Credit card numbers (16-digit patterns with optional separators)
- Aadhaar numbers (12-digit Indian national ID)
- PAN card numbers (5 letters, 4 digits, 1 letter)
- Indian mobile numbers (10-digit starting with 6-9)
- Confidentiality markers (CONFIDENTIAL, SECRET, RESTRICTED keywords)

4.2 Script Implementation

The DLP module uses Python regular expressions to scan email content. Each pattern can be tuned per organisation, and new patterns can be added without rebuilding the gateway. The full script source was saved as `dlp_demo.py` during the Day 4 practical.



```
GNU nano 9.0 dlp_demo.py
#!/usr/bin/env python3
"""DLP Demo - Scans email for sensitive data"""
import re
import sys

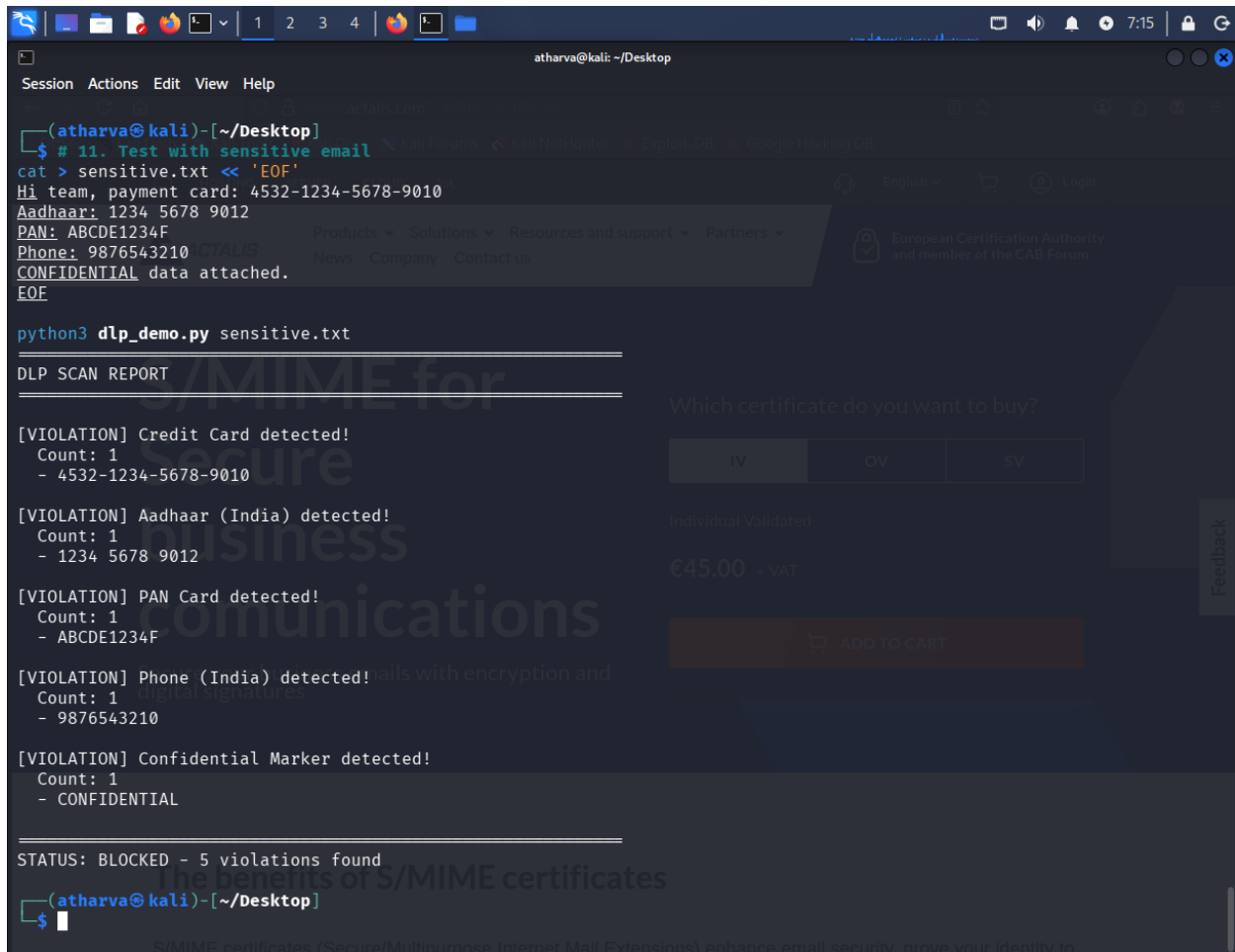
PATTERNS = {
    'Credit Card': r'\b(?:\d{4}[-\s]?){3}\d{4}\b',
    'Aadhaar (India)': r'\b\d{4}\s?\d{4}\s?\d{4}\b',
    'PAN Card': r'\b[A-Z]{5}\d{4}[A-Z]\b',
    'Phone (India)': r'\b[6-9]\d{9}\b',
    'Confidential Marker': r'\b(?:CONFIDENTIAL|SECRET|RESTRICTED)\b'
}

def scan(content):
    print("=" * 60)
    print("DLP SCAN REPORT")
    print("=" * 60)
    violations = []
    for name, pattern in PATTERNS.items():
        matches = re.findall(pattern, content, re.IGNORECASE)
        if matches:
            violations.append(name)
            print(f"\n[VIOLATION] {name} detected!")
            print(f"    Count: {len(matches)}")
            for m in matches[:3]:
                print(f"    {m}")
    print("\n" + "=" * 60)
    if violations:
        print(f"STATUS: BLOCKED - {len(violations)} violations found")
    else:
        print("STATUS: ALLOWED - Clean email")

if __name__ == "__main__":
    if len(sys.argv) > 1:
        with open(sys.argv[1]) as f:
            scan(f.read())
    else:
        pass
```

4.3 Sensitive Email Blocked

A test email was crafted containing several sensitive items (card, Aadhaar, PAN, phone) along with a confidentiality marker. The DLP scan correctly identified all violations and would block the message in a production deployment.



```
atharva@kali: ~/Desktop
Session Actions Edit View Help
$ # 11. Test with sensitive email
cat > sensitive.txt << 'EOF'
Hi team, payment card: 4532-1234-5678-9010
Aadhaar: 1234 5678 9012
PAN: ABCDE1234F
Phone: 9876543210
CONFIDENTIAL data attached.
EOF

python3 dlp_demo.py sensitive.txt

DLP SCAN REPORT

[VIOLATION] Credit Card detected!
Count: 1
- 4532-1234-5678-9010

[VIOLATION] Aadhaar (India) detected!
Count: 1
- 1234 5678 9012

[VIOLATION] PAN Card detected!
Count: 1
- ABCDE1234F

[VIOLATION] Phone (India) detected!
Count: 1
- 9876543210

[VIOLATION] Confidential Marker detected!
Count: 1
- CONFIDENTIAL

STATUS: BLOCKED - 5 violations found

(atharva@kali)-[~/Desktop]
$
```

4.4 Clean Email Allowed

A normal business meeting email passed through without any violations, confirming the filter does not interfere with legitimate communication.

atharva@kali: ~/Desktop

Session Actions Edit View Help

```
(atharva@kali)-[~/Desktop]
$ # 12. Test with clean email
cat > clean.txt << 'EOF'
Hi team, let us meet tomorrow at 10 AM.
Regards
EOF

python3 dlp_demo.py clean.txt

DLP SCAN REPORT

STATUS: ALLOWED - Clean email

(atharva@kali)-[~/Desktop]
$
```

Products Solutions Resources and support Partners

Home Company Contact us

European Certification Authority
and member of the CAB Forum

Which certificate do you want to buy?

IV

OV

SV

Individual Validated

€45.00 + VAT

ADD TO CART

Feedback

The benefits of S/MIME certificates

S/MIME certificates (Secure/Multipurpose Internet Mail Extensions) enhance email security, prove your identity to

5. Gateway Architecture

All three components work together in a pipeline. Each message is processed by every layer in turn, with the verdict from each layer recorded in the message headers for audit purposes.

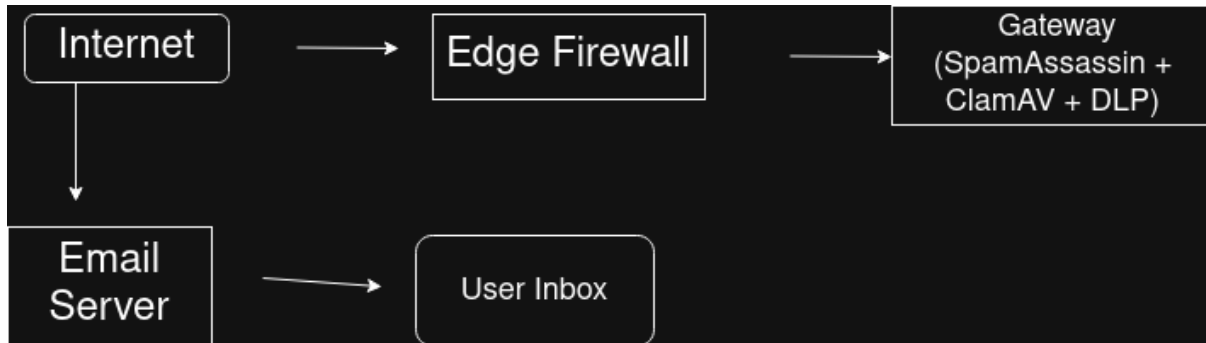


Figure: Gateway processing pipeline showing how inbound and outbound mail are inspected

5.1 Processing Order

- Inbound: Edge firewall -> ClamAV scan -> SpamAssassin score -> SPF/DKIM/DMARC verification -> Mail server
- Outbound: Mail server -> DLP scan -> ClamAV scan -> DKIM signing -> Internet

6. Operational Recommendations

- Tune SpamAssassin score threshold based on real false-positive rates after first month
- Schedule freshclam to run hourly for fast malware signature updates
- Quarantine flagged messages instead of deleting outright - allows IT team to review and release false positives
- Set up daily DLP violation reports to the security team for trend monitoring
- Enable detailed logging of all gateway decisions for forensic and compliance purposes
- Review and update DLP regex patterns quarterly as business requirements evolve

7. Conclusion

The gateway built from SpamAssassin, ClamAV, and the custom DLP module gives the consulting firm a complete inbound and outbound filtering layer at zero licensing cost. Each component was verified hands-on during this project, with screenshots throughout this deliverable serving as proof of correct operation. Combined with the encryption and authentication controls from Deliverables 2 and 3, the gateway closes the operational security gaps identified in the original infrastructure assessment.